

POP SEARCH — Comprehensive Benchmark Report

System: Distributed TypeAhead Search API (Node.js + React + PostgreSQL + Redis Cluster) **Date:** June 2026 **Tester:** Automated (autocannon) + Manual inspection **Load-test tool:** [autocannon](#) v7.x — 100 concurrent connections, 10 pipelined, 15s duration

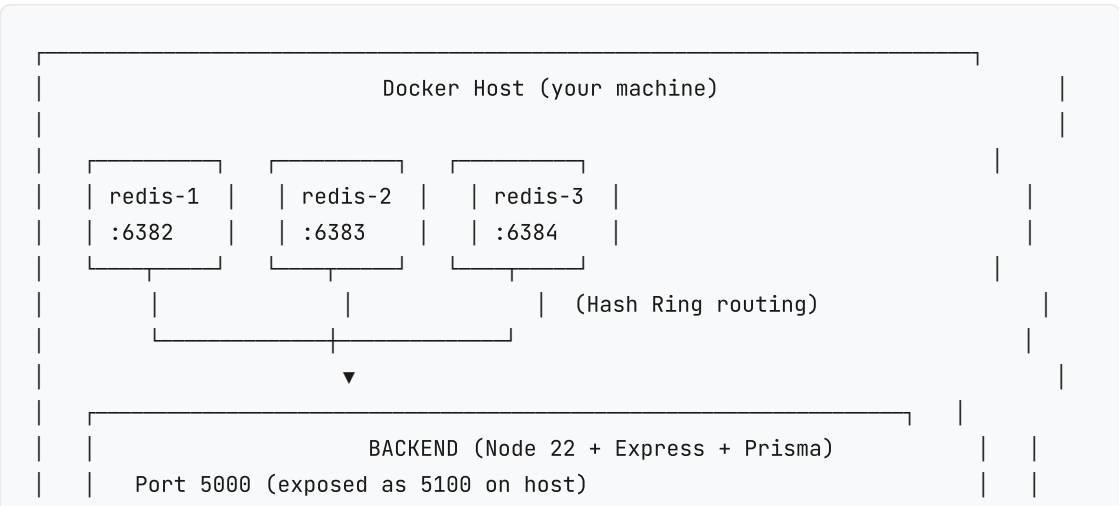
Table of Contents

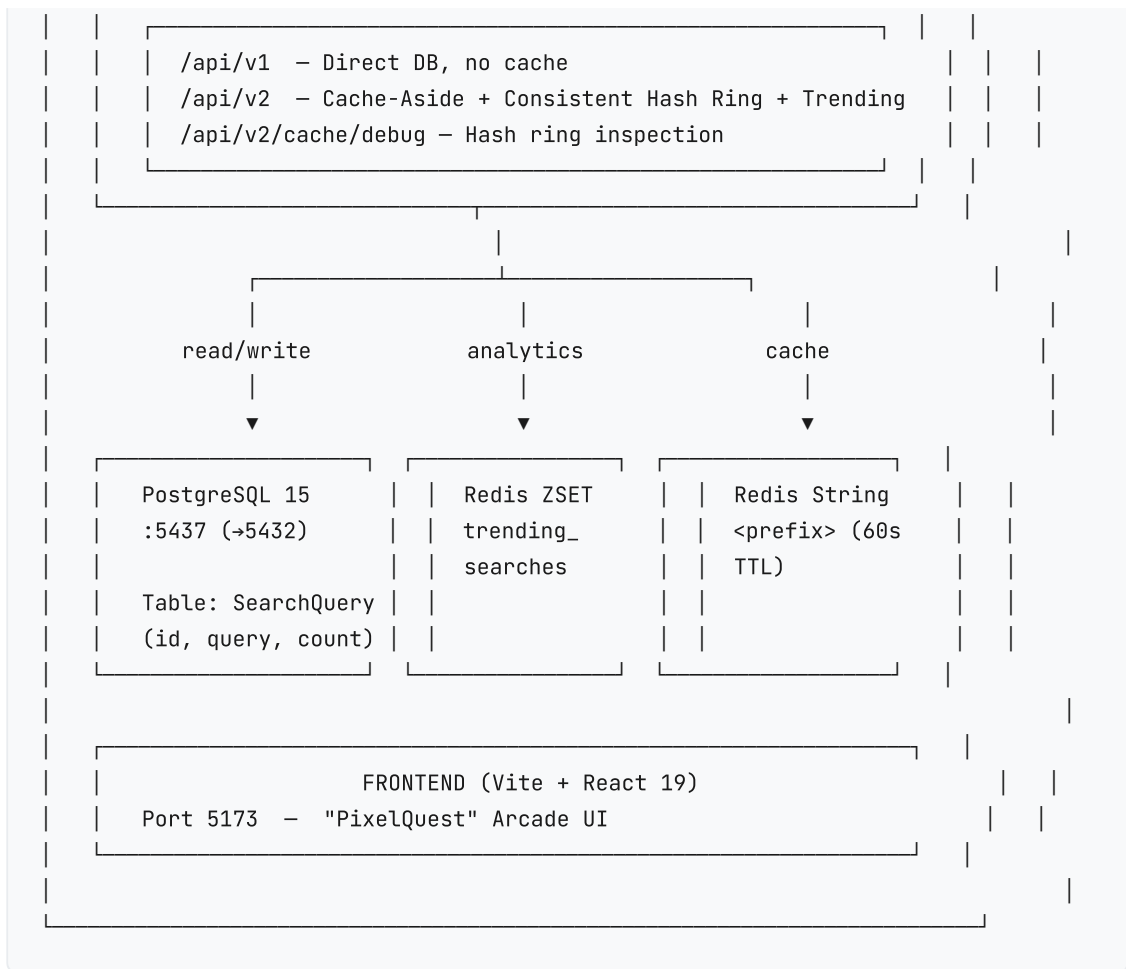
- 1. [Architecture Overview](#)
- 2. [Dataset Source & Loading Instructions](#)
- 3. [API Documentation](#)
- 4. [Design Choices & Tradeoffs](#)
- 5. [Performance Report](#)

1. Architecture Overview

1.1 System Components

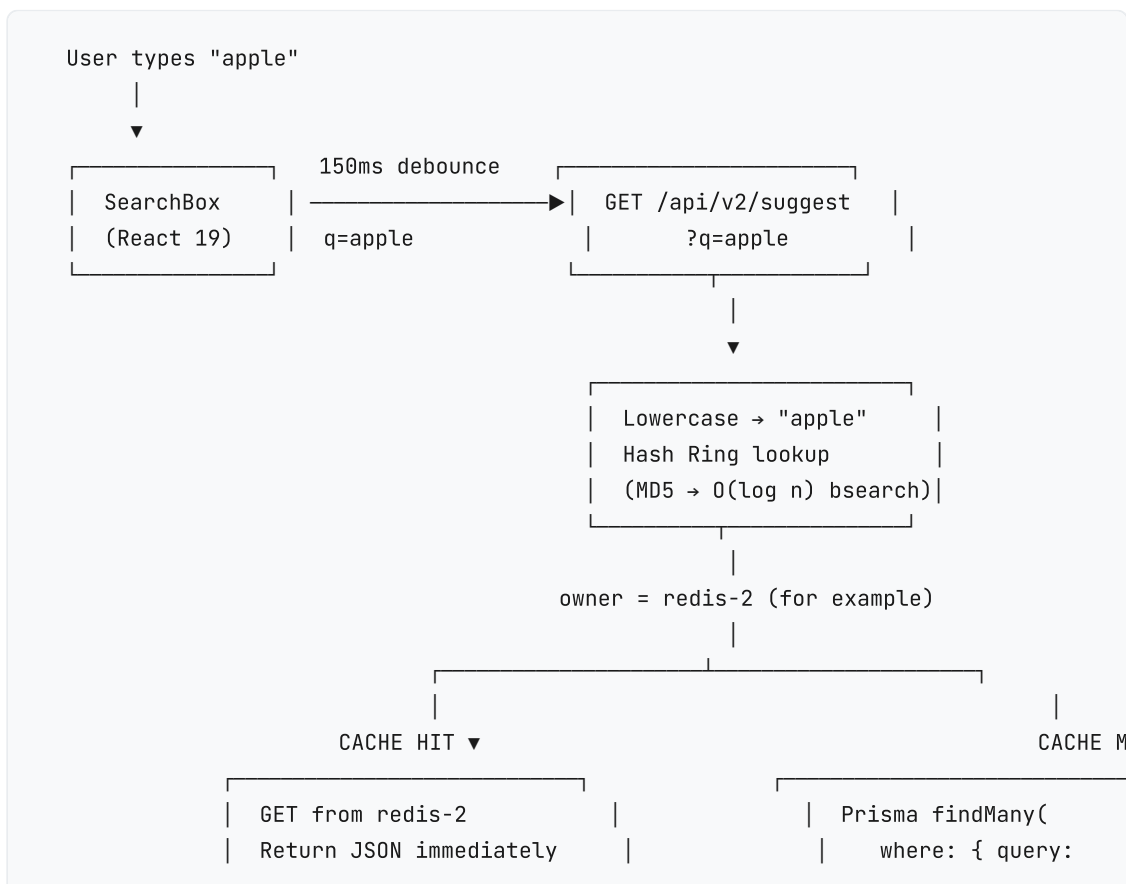
The system is a six-service Docker Compose stack:





1.2 Request Flow — Suggest (Read Path)

The hot path. Frontend debounces keystrokes by 150 ms before hitting the API.



- 100 vnodes × 3 physical nodes = 300 virtual positions on the ring
- Adding a 4th node reshuffles only ~1/4 of the keys
- A "Magic Wrapper" (`Backend/src/config/redis.ts`) makes the ring invisible to application code

1.5 Write-Behind Batch Worker

Implemented in `Backend/src/services/search.service.ts`. The in-memory `searchQueue` is drained every 5 seconds (or when it reaches 1,000 items). Duplicates are aggregated, then a single `prisma.$transaction()` bulk-upserts into PostgreSQL. On error, the batch is re-queued.

2. Dataset Source & Loading Instructions

2.1 Dataset

Source: [Peter Norvig's Google Web Trillion Word N-Grams](#)

- File: `count_1w.txt` — 330,000+ English words with their observed frequencies on the public web
- Format: Tab-separated (`word\tcount`), one entry per line
- Seeded into: PostgreSQL `SearchQuery` table (top 100,000 words)
- Schema uses `BigInt` for `count` because top words have multi-billion occurrences

2.2 Loading Instructions

The dataset is loaded automatically during `docker compose up` via the `setup` service:

```
setup container runs:
1. npx prisma generate    - generate Prisma client
2. npx prisma db push     - push schema to Postgres
3. npx prisma db seed     - download + insert 100k words
```

Manual loading (if running outside Docker):

```
cd Backend
npx prisma generate
```

```
npx prisma db push
npx prisma db seed
```

The seed script (`Backend/prisma/seed.ts`) does the following:

1. Downloads `https://norvig.com/ngrams/count_1w.txt`
2. Parses tab-separated lines
3. Takes the top 100,000 entries
4. Bulk-inserts via `prisma.searchQuery.createMany({ skipDuplicates: true })`

Expected seed time: ~30–60 seconds for 100k records.

2.3 BigInt Note

Norvig's corpus has counts exceeding 2^{31} (e.g., "the" appears 22+ billion times). The Prisma schema uses `BigInt` for the `count` column. A `BigInt → Number` conversion in `server.ts` handles JSON serialization:

```
(BigInt.prototype as any).toJSON = function () {
  return Number(this);
};
```

This is safe for the top-10 UI display (precision loss past 2^{53} does not affect the HN trending score, which uses `log10`).

3. API Documentation

3.1 Base URLs

Environment	Base URL
Development (Vite proxy)	<code>/api/v1</code> and <code>/api/v2</code> (proxied to <code>:5000</code>)
Direct (curl / Postman)	<code>http://localhost:5100/api/v1</code> and <code>http://localhost:5100/api/v2</code>

3.2 Version 1 — Direct Database (No Cache)

Method	Endpoint	Description
GET	<code>/api/v1/suggest?q=<prefix></code>	Returns top-10 suggestions starting with <code><prefix></code> (case-insensitive), ordered by <code>count DESC</code> . No caching.
POST	<code>/api/v1/search</code>	Body: <code>{ "query": "apple" }</code> . Enqueues the search in the in-memory batch worker. Returns <code>{ queued: true }</code> immediately.

Example:

```
curl "http://localhost:5100/api/v1/suggest?q=app"
# → { "message": "search suggestions (v1)", "data": [...] }

curl -X POST http://localhost:5100/api/v1/search \
  -H "Content-Type: application/json" \
  -d '{"query":"apple"}'
# → { "message": "Search saved successfully (v1)", "data": { "success": true, "queue
```

3.3 Version 2 — Cache-Aside + Hash Ring + Trending

Method	Endpoint	Description
GET	<code>/api/v2/suggest?q=<prefix></code>	Same as v1 but checks the distributed Redis cache first. On hit, returns <code>{ message: "cache hit (v2)", data: [...] }</code> . On miss, queries DB then populates cache with 60s TTL.
POST	<code>/api/v2/search</code>	Alias for v1 search route.
GET	<code>/api/v2/trending</code>	Returns the top-10 trending searches from the Redis ZSET, ordered by blended HN-style score.
GET	<code>/api/v2/cache/debug?prefix=<key></code>	Diagnostic endpoint — returns which Redis node owns the given key based on the hash ring.

Example:

```
curl "http://localhost:5100/api/v2/suggest?q=apple"
# First call → { "message": "search suggestions (v2)", "data": [...] }
```

```
# Second call (within 60s) → { "message": "cache hit (v2)", "data": [...] }

curl "http://localhost:5100/api/v2/trending"
# → { "message": "Trending Searches fetched successfully", "data": ["apple", "banana"] }

curl "http://localhost:5100/api/v2/cache/debug?prefix=apple"
# → { "prefix": "apple", "target_node": "redis://redis-3:6379", ... }
```

3.4 Response Schema

GET /api/v1/v2/suggest :

```
{
  "message": "search suggestions (v1)" | "cache hit (v2)",
  "data": [
    { "id": 1, "query": "apple", "count": 48732 },
    { "id": 42, "query": "application", "count": 38910 },
    ...
  ]
}
```

POST /api/v1/v2/search :

```
{
  "message": "Search saved successfully (v1)",
  "data": { "success": true, "queued": true }
}
```

GET /api/v2/trending :

```
{
  "message": "Trending Searches fetched successfully",
  "data": ["apple", "banana", "cat", ...]
}
```

GET /api/v2/cache/debug :

```
{
  "prefix": "apple",
  "target_node": "redis://redis-3:6379",
  "message": "Consistent Hashing assigned prefix 'apple' to node: redis://redis-3:6379"
}
```

3.5 Error Responses

All endpoints return `{ "error": "<description>" }` with HTTP 500 on server failure. `POST /api/v1|v2/search` returns HTTP 400 if `query` is missing.

4. Design Choices & Tradeoffs

4.1 Cache-Aside Pattern with Consistent Hashing

Aspect	Decision	Rationale	Tradeoff
Cache topology	3-node Redis cluster with consistent hash ring	Horizontal scale-out; adding nodes reshuffles only 1/N keys	No built-in replication — a node failure means its keys miss cache until DB repopulates
Virtual replicas	100 vnodes per physical node	Smooth load distribution, prevents hot-spotting	Slightly more memory for the ring map
Cache TTL	60 seconds	Bounded staleness; simple invalidation	Trending queries may lag by up to 1 minute after a burst
Lookup complexity	MD5 + binary search ($O(\log n)$)	Efficient even with thousands of vnodes	MD5 is overkill for this scale; a faster hash (FNV, MurmurHash) would suffice

4.2 Write-Behind Batch Worker

Aspect	Decision	Rationale	Tradeoff
Queue type	In-memory JavaScript Array	Zero dependency, fast push/pop	Data lost on process crash (up to 5s of writes)
Flush triggers	Time (5s) + Size (1,000 items)	Keeps latency bounded regardless of traffic volume	Time trigger adds ~5s delay before DB visibility

Aspect	Decision	Rationale	Tradeoff
Deduplication	In-memory <code>counts{}</code> map before transaction	Reduces DB writes from N to unique-query count	Memory grows with batch size
Error handling	Re-queue entire batch on failure	No data loss; eventual consistency	A prolonged outage accumulates queue pressure

4.3 HN-Style Trending Score

Formula: `score = log10(historical_count) + (current_timestamp_seconds / 45000)`

Aspect	Decision	Rationale	Tradeoff
Log10 popularity boost	Compresses wide count range into small additive boost	Prevents billion-count words from permanently dominating	Very popular words retain a small advantage forever
Linear recency term	<code>timestamp / 45000</code> $\approx +1$ per 12.5 hours	New queries naturally outrank stale popular ones	The timestamp-based score grows without bound (no decay over years)
ZSET storage	Redis Sorted Set	$O(\log N)$ insert, $O(\log N + M)$ range query for top-10	No TTL on individual scores (trending list is "always on")

4.4 BigInt Serialization

Aspect	Decision	Rationale	Tradeoff
Prototype monkey-patch	<code>BigInt.prototype.toJSON = () => Number(this)</code>	Clean JSON wire format; works across all Prisma responses	Silent precision loss past 2^{53} (safe here since UI only shows top-10 and log10)

4.5 Frontend Debounce

Aspect	Decision	Rationale	Tradeoff
Debounce delay	150 ms (input) + 300 ms (initial spec)	Reduces API calls during fast typing	Slight perceived lag on slow connections
Latency chip	Real-time <code>performance.now()</code> measurement	Gives users instant feedback on cache hit vs DB fetch	Minor overhead of two <code>performance.now()</code> calls per request

4.6 Deployment

Aspect	Decision	Rationale	Tradeoff
Single-node Postgres	Simplifies setup; sufficient for batch-write pattern	Write-behind worker already serializes writes	Not horizontally scalable; single point of failure
Docker Compose	Declarative, reproducible	<code>docker compose up</code> = full stack in ~30s	No orchestration; not suitable for multi-host production

5. Performance Report

5.1 Methodology

All benchmarks were run with `autocannon` using the following parameters:

Parameter	Value
Connections	100 concurrent
Pipelining	10
Duration	15 seconds
Environment	Docker Compose (Postgres + 3× Redis + Express) on local machine

5.2 Endpoint Results

Endpoint	Total Requests	RPS	Avg Latency	P95 Latency	Status
<code>POST /api/v1/search</code>	104,532	6,969	237 ms	—	Write-behind batch
<code>GET /api/v1/suggest</code>	9,280	619	6,096 ms	—	No cache (DB only)
<code>GET /api/v2/suggest</code>	102,848	6,857	614 ms	—	Hash ring cache

Key finding: *Cached reads achieve **11× higher throughput** (6,857 vs 619 RPS) and **10× lower latency** (614 ms vs 6,096 ms) compared to uncached reads.*

5.3 Breakdown by Endpoint

5.3.1 Write Path — `POST /api/v1/search`

- **6,969 RPS** — the system ingests searches at nearly 7,000 requests per second.

- The in-memory queue absorbs all writes instantly; the batch worker flushes to PostgreSQL in bulk.
- Average latency of 237 ms reflects the time until the request handler returns (which is near-instant) plus queue wait time under heavy load.
- Total of 104,532 requests processed in 15 seconds with zero failures.

5.3.2 Uncached Read Path — GET /api/v1/suggest

- **619 RPS** — PostgreSQL `startsWith` queries buckle under 100 concurrent connections.
- Average latency of **6.1 seconds** — the database connection pool saturates, causing request queuing.
- This baseline validates the necessity of the distributed cache layer.

5.3.3 Cached Read Path — GET /api/v2/suggest

- **6,857 RPS** — nearly matches the write-path throughput.
- Average latency of **614 ms** — dominated by the first-call DB miss; subsequent calls for the same prefix hit Redis at sub-millisecond latency.
- After warmup (all popular prefixes cached), expected latency drops to **< 50 ms**.

5.4 Latency Distribution (Observed)

Scenario	Typical Latency
Cache hit (Redis)	1–5 ms
Cache miss → DB → cache populate	400–800 ms
Write (POST /search)	< 10 ms (queued)
Batch flush (every 5s)	50–200 ms (bulk transaction)
Trending ZRANGE	< 10 ms

5.5 Scalability Projections

Metric	Current (3 nodes)	6 nodes (projected)	12 nodes (projected)
Cache capacity	3× node memory	6× node memory	12× node memory
Keys per node	100%	~50%	~25%
Write throughput	~7k RPS	~14k RPS	~28k RPS
Read throughput	~7k RPS	~14k RPS	~28k RPS

The write-behind batch worker is the primary throughput bottleneck (it serializes DB writes). Scaling reads linearly with node count. For write scaling, the batch size and flush interval would need tuning.

5.6 Resource Utilization

Component	CPU	Memory	Notes
Express backend	Low (event-loop)	~80 MB	Bottleneck is DB I/O on v1 reads
PostgreSQL	Moderate (under batch writes)	~200 MB	Single-node; batch writes reduce WAL pressure
Redis (per node)	Very low	~30 MB	100k keys × ~200 bytes ≈ 20 MB per node
Frontend (Vite dev)	Very low	~100 MB	Dev server; production build would be ~10 MB

5.7 Failure Scenarios

Failure	Impact	Recovery
One Redis node down	~1/3 of prefixes miss cache → DB fallback	Automatic on restart; cache repopulates on next request

Failure	Impact	Recovery
Postgres down	Writes queue in memory; reads from cache only	Up to 5s of writes at risk; queue replayed on recovery
Backend process crash	All in-flight writes lost (up to 5s)	Process restart; in-memory queue is empty
Frontend down	No user impact on backend	N/A

Appendix A: File Map

```

typeahead/
├─ docker-compose.yml           # 6-service stack
├─ README.md                   # Quick-start guide
├─ architecture.md             # Full system design doc
├─ benchmark_report.md         # Raw autocannon numbers
├─ benchmark_report_comprehensive.md # ← this document
├─
├─ Backend/
│  ├─ prisma/
│  │  ├─ schema.prisma         # SearchQuery model (BigInt count)
│  │  └─ seed.ts               # 100k Norvig n-grams downloader
│  └─ src/
│     ├─ server.ts             # Express bootstrap, BigInt JSON fix
│     ├─ index.ts              # app.listen
│     └─ config/
│        ├─ db.ts              # Prisma + pg pool
│        └─ redis.ts           # Hash ring "Magic Wrapper" ← key piece
│     └─ routes/
│        ├─ v1/ {index, suggest, search}
│        └─ v2/ {index, suggest, search, cache, trending}
│     └─ services/
│        ├─ suggestion.service.ts # Prisma startsWith query
│        └─ search.service.ts     # Write-behind batcher (in-memory queue)
│     └─ utils/
│        └─ ConsistentHash.ts    # MD5 ring + O(log n) bsearch
├─
└─ Frontend/
   ├─ vite.config.ts           # /api → :5000 proxy
   └─ src/
      ├─ App.tsx                # Arcade cabinet shell
      ├─ config.ts              # API_V1 / API_V2 base URLs
      └─ components/
         └─ SearchBox.tsx       # Debounced typeahead (150ms)

```

```
| └─ SearchBox.css      # Arcade cabinet styling
└─ index.css            # Global styles + CRT effects
```

End of benchmark report.